

毕业设计答辩讲稿

2026-05-25

毕业设计答辩讲稿参照

本文档对应当前 ppt/finalterm/2022112879-郑彦文-毕业设计答辩.pptx。当前答辩演示文稿共 20 页，其中第 1-15 页为正式汇报，第 16-19 页为备份页，第 20 页为结束页。正式汇报建议控制在 8-10 分钟，目标时长约 9 分钟 30 秒，问答约 5 分钟。

使用时不要逐字背诵。每页只抓住一个问题：本页回答什么、屏幕上看哪里、我想让老师记住哪一句、如果被追问再补充什么。

术语口播约定

答辩时尽量先说中文名，必要时再补英文或代码字段名。后文代码对象为了和演示文稿、实现字段对应，会保留英文标识。

- CEBRA-WP：约束与证据感知、信念引导、恢复自适应的工作流规划。
- LLM：大语言模型；Agent：智能体。
- ToolKG / ProteinToolKG：工具知识图谱 / 蛋白质工具知识图谱。
- schema：输入输出字段模式；I/O：输入/输出闭合。
- feasibility：候选可执行性，口播时可解释为“这条工具链能不能接上、能不能安全地执行”；论文中对应硬可行性过滤。
- score_breakdown：评分分解；static_score / S_static：静态评分或静态效用；posterior_objective / G_post：后验目标匹配。
- S_post：后验评分；clip：截断函数，表示把分数限制在指定范围内。
- Lite belief-state：轻量信念状态，也可以口播为“五维运行时状态摘要”。

- RuntimeState：运行时状态；StepResult：步骤执行结果；SafetyResult：安全检查结果。
- FSM：有限状态机；HITL：人在环决策；Workflow/FSM：工作流/有限状态机控制。
- PendingAction：待决策对象；Decision：决策记录；EventLog：事件日志；Snapshot / TaskSnapshot：快照 / 任务快照。
- continue、patch_local、suffix_replan、stop：继续执行、局部修补、后缀重规划、受保护停止。
- ActionUtility：动作效用；RuntimeEvaluator：运行时评估器；RecoveryAction：恢复动作。
- budget pressure：预算压力；evidence_sufficiency：证据充分度。
- Top-K：保留排名靠前的 K 个候选。
- POMDP：部分可观测马尔可夫决策过程；RL：强化学习。

一、整体主线

本次答辩的主线是：

高代价、可失败的蛋白质设计工具链，如何在执行过程中做可解释、可恢复、可审计的规划与恢复决策。

需要让老师记住四句话：

1. 本文贡献集中在运行时工作流规划，不是训练新的蛋白质生成模型。
2. CEBRA-WP 先用工具知识图谱、字段模式、输入/输出闭合、安全和预算做硬过滤，再对合法候选做评分和重排。
3. 运行时自适应层用步骤执行结果、安全检查结果、失败历史和预算进度更新轻量信念状态。
4. 算法输出继续执行、局部修补、后缀重规划、受保护停止四类动作，并通过有限状态机/人在环决策、事件日志和快照进入可审计流程。

二、时间分配

页码	页面主题	建议时间	讲清楚的事
1	封面	10 秒	题目与工作定位
2	目录	15 秒	给出汇报顺序，提醒 算法与验证是重点
3	汇报结构与核心问题	20 秒	本文围绕一个运行时 控制问题展开

页码	页面主题	建议时间	讲清楚的事
4	为什么需要 workflow 规划机制	40 秒	工具链长、贵、会失败，需要控制层
5	系统架构如何支撑算法落地	45 秒	架构简讲，补充五层职责与算法位置
6	算法接口链	35 秒	模块之间传结构化对象，不传自然语言计划
7	有限状态机/人在环决策约束	35 秒	有限状态机和人在环决策约束算法动作
8	CEBRA-WP 总览	50 秒	静态规划层 + 运行时自适应层
9	静态规划	55 秒	候选生成、硬过滤、静态评分
10	轻量信念状态	55 秒	稀疏观测如何变成五维轻量信念状态
11	动态决策	55 秒	有界重排和四类恢复动作
12	算法落到代码	40 秒	理论对象对应实现字段和模块
13	系统测试验证	35 秒	工程路径是否稳定、可恢复、可审计
14	84 次运行消融	55 秒	算法机制可观测，成本控制有证据
15	贡献与边界	25 秒	收束贡献，不夸大结论

备份页第 16-19 页只在问答时使用，不主动展开。

三、逐页讲稿

第 1 页：封面

回答的问题：这项毕业设计做的是是什么。

建议口播：

各位老师好，我是软件工程专业郑彦文。我的毕业设计题目是《基于大模型驱动的 Agent 协作新一代蛋白质设计系统开发》，这里的 Agent 指智能体。这项工作关注的不是重新训练一个蛋白质生成模型，而是面向已有蛋白质设计工具链，构建一个可规划、可执行、可恢复、可审计的工作流控制系统。

转场：

下面先看本次汇报的目录安排。

第 2 页：目录

回答的问题：整场汇报按什么顺序展开。

屏幕重点：五个目录项，尤其是“核心算法”和“验证结论”。

建议口播：

这页给出本次汇报的顺序。前两部分先说明为什么需要 workflow 规划机制，以及系统架构如何约束算法动作；中间重点讲 CEBRA-WP，也就是约束与证据感知、信念引导、恢复自适应的工作流规划，包括候选过滤、静态评分、轻量信念状态和恢复动作选择；后面说明这些算法对象如何落到实现，并用系统测试和 84 次运行消融实验验证机制是否成立。

转场：

下面把本次答辩围绕的核心问题压缩成一句话。

第 3 页：汇报结构与核心问题

回答的问题：本次答辩到底围绕哪条线展开。

屏幕重点：中间五个阶段，尤其是蓝色的“核心算法”。

建议口播：

本次答辩围绕一个问题展开：高代价、可失败的蛋白质设计工具链，如何在执行过程中做可解释的规划与恢复决策。前两部分先说明工程约束和系统支撑，中间重点讲核心算法 CEBRA-WP，也就是约束与证据感知、信念引导、恢复自适应的工作流规划，后面再说明它如何落到代码，以及用哪些测试和消融实验验证。

这里的核心判断是，本文的贡献集中在运行时工作流规划：先约束候选可执行，再依据运行时证据选择继续执行、局部修补、后缀重规划或受保护停止。

补充说明：

这页不要展开算法细节，只给老师建立“后面主要看算法机制和工程落地”的预期。

转场：

先看为什么蛋白质设计场景需要这样的工作流规划机制。

第 4 页：为什么需要工作流规划机制

回答的问题：为什么不能只把几个工具顺序调用起来。

屏幕重点：接口异构、高代价调用、失败可恢复、决策需审查。

建议口播：

蛋白质设计工具链不是一次模型调用，而是生成、结构预测、质量门禁、目标评分和结果汇总组成的长链路。工程上有四个问题。第一，序列、结构、评分工具的输入输出格式差异明显。第二，结构预测和重型打分属于高代价调用，证据不足时不应该盲目执行。第三，参数错误、输入/输出缺失、质量门禁失败都可能出现，但这些失败不一定意味着任务必须终止。第四，高风险、高成本或不确定的决策需要人工确认。

本文的边界也在这页说明清楚：我不训练新的蛋白质生成模型，不宣称湿实验验证，也不替代 AlphaFold、ESMFold 等底层结构预测工具。我的切入点是在已有工具之上实现工作流控制层。

可以补充的难点：

难点不在于把工具简单串起来，而在于失败后如何判断：是继续、局部修补、后缀重规划，还是受保护地停止。这个判断必须同时考虑工具约束、证据、成本和安全边界。

转场：

有了问题边界，下面看系统架构如何承载这个控制层。

第 5 页：系统架构如何支撑算法落地

回答的问题：CEBRA-WP 在五层控制架构中的位置，以及规划、执行、审计如何被工作流/有限状态机统一约束。

屏幕重点：先看从上到下的五层，再看右侧“控制面单一事实源”和“可恢复审计链”，最后看底部工具资源。

建议口播：

这页展示的是系统的整体架构。它不是单纯把模块堆在一起，而是说明大模型规划如何被收束到一个可控制、可恢复的工程流程里。

从上到下看，第一层是输入与交互层，包括命令行接口、后端接口和前端工作台，负责把用户需求整理成任务，并承接人工确认。第二层是契约与状态控制层，里面有蛋白质设计任务、已确认任务规格、任务记录和工作流上下文；这里是系统的控制面，状态变化由工作流/有限状态机统一管理。第三层是智能规划层，规划智能体、候选生成器和蛋白质工具知识图谱在这里生成 Top-K 候选计划、局部修补候选或重规划候选，并给出候选可执行性检查和评分分解，CEBRA-WP 主要工作在这一层。第四层是执行与恢复层，由执行智能体、计划运行器、修补运行器和步骤运行器负责步骤派发、依赖解析、有限重试，以及局部修补、后缀重规划这类恢复动作。第五层是安全、汇总与资源层，包括安全智能体、汇总智能体、适配器注册表、工具适配器、事件日志、任务快照和最终产物。

右侧两块是这张图的关键。控制面单一事实源表示只有工作流/有限状态机可以修改任务状态，进入等待态时执行会暂停，等待人工决策或恢复动作确认。可恢复审计链表示在进入等待状态前，系统会写入待决策对象、决策记录、事件日志、任务快照和运行时状态，因此失败后可以从快照恢复，也可以回放当时为什么做这个决策。

所以 CEBRA-WP 的定位不是直接执行蛋白质设计工具，而是在规划层输出候选计划和默认建议。真正执行时，候选必须回到状态控制层，经由确认后的计划或快照恢复，再进入执行层。这样可以避免规划智能体直接越权调用工具，也能让每一次修补和重规划都有证据可查。

可以补充的难点：

这里的工程难点主要有两点。第一是控制面分离：规划智能体只产出候选，不直接执行；执行智能体只执行已确认步骤，不越过状态机；安全智能体只输出风险信号，不直接改计划；汇总智能体只汇总已有证据，不重新计算。第二是工具异构性：底部资源层包含序列生成、结构预测、质量检查、目标评分和可视化工

具，这些工具的输入输出、成本、风险和可用性都不同。因此系统用蛋白质工具知识图谱记录工具能力、输入/输出字段模式、成本、安全等级和兼容关系，再由适配器注册表映射到具体后端，这也是后面算法能够做硬过滤和运行时决策的基础。

转场：

因此下一页不展开完整运行时序，而是看这些层之间实际传递的结构化信息。

第 6 页：算法接口链：模块之间传递什么

回答的问题：模块之间如何传递信息，讲到哪一步即可。

屏幕重点：上方规划与确认链，下方执行结果、运行时状态与恢复动作链。

建议口播：

模块之间的信息传递可以理解为算法接口链。系统不是让智能体直接互相传自然语言计划，而是把算法所需信息收束成结构化对象。

上方这条链是规划侧：任务先被表示为任务规格，包含目标、约束和预算；规划器生成候选集合，包括计划、局部修补和重规划候选；候选经过“候选可执行性”和“评分”两类判断，得到阻断原因和评分分解；需要确认的候选进入待决策对象，最后形成决策记录。

下方这条链是运行时侧：执行器执行后产生步骤执行结果，安全智能体产生安全检查结果，这些观测汇入运行时状态；运行时评估器再输出恢复动作，最后通过事件日志和快照留下审计与恢复证据。

可以补充的难点：

难点在于结构化契约要同时服务三件事：让算法能计算，让执行器能运行，让日志和快照能回放。自然语言解释可以辅助展示，但不能作为模块间的真实控制接口。

转场：

有了结构化对象，还需要状态机约束算法动作，避免恢复动作越权。

第 7 页：有限状态机/人在环决策：恢复动作如何被约束

回答的问题：算法输出动作后，系统如何保证它可审查、可追溯。

屏幕重点：左侧状态迁移图中的等待态，右侧三类说明框。

建议口播：

这页说明 CEBRA-WP 的输出如何被有限状态机和人在环决策约束。图中的虚线区域对应人工确认边界。系统中有几类关键等待态：初始计划确认、局部修补确认、重规划或停止候选确认。进入等待态后，执行器不继续调用工具，而是等待待决策对象被确认。

算法输出继续执行、局部修补、后缀重规划和受保护停止后，并不是直接改变底层工具执行，而是转成决策记录、事件日志和快照。这样每个恢复动作都可以被审查、记录和恢复。

可以补充的难点：

尤其是受保护停止动作不能由算法静默执行，因为它代表放弃当前任务或进入终止路径。本文把它作为终止型停止候选进入重规划确认和人在环决策，保证高风险动作有人工确认边界。

转场：

到这里，系统边界已经说明清楚。下面进入核心算法 CEBRA-WP。

第 8 页：CEBRA-WP：静态规划层 + 运行时自适应层

回答的问题：CEBRA-WP 到底是什么。

屏幕重点：上方四步流程和下方“输入、核心状态、输出”。

建议口播：

CEBRA-WP 可以分成两层。第一层是静态规划层，它在执行前根据任务目标、约束和工具知识图谱生成候选，做硬可行性过滤，并建立静态评分。第二层是运行时自适应层，它在执行过程中读取步骤执行结果、安全检查结果、失败历史和预算信息，更新轻量信念状态，再对候选和恢复动作做有界调整。

这套机制的输入包括任务、约束、工具知识图谱、运行时上下文和策略；核心状态是五维运行时状态；输出是继续执行、局部修补、后缀重规划和受保护停止。

可以补充的难点：

这里最重要的边界是：运行时状态只修正已经合法的候选，不覆盖工具存在性、字段模式、输入/输出闭合、安全和预算这些硬约束。也就是说，运行时证据可以改变优先级，但不能把不可执行候选重新放回执行队列。

转场：

先看执行前的静态规划层如何保证候选可执行。

第 9 页：静态规划：候选、硬过滤与静态评分

回答的问题：执行前如何保证候选工具链可执行，并建立先验排序。

屏幕重点：上方五步链路和底部 S_{static} 。

建议口播：

静态规划层先回答“哪些候选能执行”，再回答“哪条链先验上更值得尝试”。输入是任务目标、预算、允许或禁用工具，以及蛋白质工具知识图谱中的工具能力和输入/输出信息。候选生成后，系统先做硬可行性过滤，也可以口播成“候选可执行性检查”：检查工具是否存在、字段模式是否匹配、跨步骤输入/输出是否闭合、是否违反安全规则或预算上限。

通过硬过滤后，系统再用目标匹配、成本、风险、恢复复杂度和工程可靠性等因素形成静态评分 S_{static} ，并保留 Top-K 多样候选。这样做可以避免把不可执行候选带入运行时阶段，也避免一次性押注在单条大语言模型生成链路上。

可以补充的难点：

大语言模型生成的候选可能“看起来合理”，但实际上字段模式不对、输入输出接不上，或者需要的工具不可用。因此硬过滤必须是确定性的，安全和预算也不能被高目标分抵消。静态评分只在候选合法之后生效。

转场：

静态规划解决执行前的问题，执行过程中还需要处理稀疏、不完整的运行时观测。

第 10 页：轻量信念状态：把稀疏观测压缩成可解释状态

回答的问题：执行中的失败、风险和成本信息如何被算法使用。

屏幕重点：左侧五类观测，中间轻量信念状态，右侧五个状态变量。

建议口播：

运行时阶段的难点是观测不完整。一次工具失败不一定说明整条路线都错了，一次成功也不能说明后续一定可靠。因此系统没有把运行时事件直接变成动作，而是先更新一个轻量信念状态，也就是多维运行时状态摘要。

这个状态来自五类信息：步骤执行结果中的输出、指标和错误；安全检查结果中的风险标记；失败上下文和修补/重规划历史；预算与任务进度；以及人在环决策记录。它被压缩成五个变量： $p_success$ 表示继续成功概率， $p_structural_failure$ 表示结构性失败压力， $recovery_margin$ 表示恢复余量， $expected_remaining_cost$ 表示预期剩余成本， $evidence_sufficiency$ 表示证据充分度，也就是证据是否足以进入高代价步骤。

可以补充的难点：

这不是完整的部分可观测马尔可夫决策过程，也不是学习得到的最优策略，而是受信念状态规划启发的工程化低维状态代理。当前实现采用确定性更新规则表：成功会提高继续成功概率和恢复余量，失败会提高结构性失败压力和剩余成本，安全阻断会作为强负证据。这样做的好处是可解释、可序列化、能进入快照，也能在实验中观察到。

转场：

有了运行时状态，下一步是用它影响候选重排和恢复动作选择。

第 11 页：动态决策：重排候选并选择恢复动作

回答的问题：运行时状态如何影响下一步动作。

屏幕重点：上方候选运行时效用 U_pi 和动作效用，下方四类动作。

建议口播：

动态阶段有两个输出。第一个是候选重排：静态分或后验目标分作为基础，运行时状态只做有界修正，也就是 $U_pi = clip(S_post + Delta, 0, 1)$ 。这里的 $Delta$ 可以口播为“运行时修正项”， $clip$ 表示把分数限制在 0 到 1 之间；它们不会改变硬过滤结果，只影响已通过硬过滤候选的排序。

第二个输出是恢复动作选择。动作效用会根据当前状态、预算、恢复余量和候选形状，在四类动作中选择：成功概率和证据充分度尚可时继续执行；故障局部化且恢复余量足时局部修补；结构性失败压力高、当前后缀不可靠时后缀重规划；成功率低、预算压力高时受保护停止，但停止动作会进入人在环决策。

可以补充的难点：

运行时信号容易被单次失败误导，所以运行时修正项必须有界；高代价调用不能盲目重跑，所以预算压力和证据充分度会抑制无效调用；停止动作风险最高，所以实现中它不直接终止任务，而是进入重规划确认和人在环决策。

转场：

算法讲清楚之后，还要证明它不是停留在概念图里，而是落到了代码对象和模块中。

第 12 页：算法对象如何落到代码

回答的问题：算法对象在实现里对应什么字段和模块。

屏幕重点：表格中的三列：算法对象、实现字段/模块、作用。

建议口播：

这一页用来说明算法已经工程落地。硬可行性过滤落在 `candidate_feasibility` 和候选生成器中，口播时可以说“候选可执行性字段”，它用来阻断工具不存在、字段模式不匹配、输入/输出不闭合、安全或预算违规的候选。静态评分落为 `score_breakdown.overall` 和 `static_score`，也就是评分分解和静态评分。后验目标落为 `posterior_objective`，表示基于运行时证据更新后的目标匹配。轻量信念状态落为 `RuntimeState` 和 `belief_state.py`。运行时修正由 `runtime_adjustment` 和 `RuntimeEvaluator` 完成。动作效用由 `ActionUtility` 计算。

最后，算法结果不会绕过系统直接执行，而是进入待决策对象、决策记录、事件日志和快照。这也是它可审计、可恢复的实现基础。

可以补充的架构说明：

如果老师追问代码结构，可以补充：接口层负责暴露任务、事件、报告和人工决策接口；工作流层负责有限状态机、计划运行器、步骤运行器和运行时评估器；适配器层封装本地或远程工具；存储层保存事件日志和快照。前端只展示后端状态和决策入口，不自行推导状态机。

转场：

实现之后，需要用测试和实验说明机制是否真的可运行、可观察。

第 13 页：验证一：系统测试覆盖关键路径

回答的问题：系统本身是否稳定可运行，关键路径是否有工程验证。

屏幕重点：上方 13、12、1、5、0 五个数字；中间五类覆盖路径；底部工程验证结论。

建议口播：

验证我拆成两页讲。第一页回答工程问题：这个系统是不是能稳定跑起来，关键状态和恢复路径是否可审计。

这里一共设计了 13 个系统测试用例，12 个通过，1 个命令行相关用例部分通过，没有出现静默崩溃。中间五个框对应覆盖范围：接口和任务录入，有限状态机、人在环决策与快照，前端和命令行，端到端工具链，以及异常、安全和恢复路径。

这一页的重点不是证明算法优于基线，而是证明系统具备实验前提：任务能创建，计划能执行，等待态能停住，人工决策能被记录，失败和恢复路径能留下事件日志与快照。

结论边界：

系统测试说明工程链路是可运行、可恢复、可审计的；算法说服力主要放在下一页 84 次运行消融实验。

转场：

工程路径可用之后，再看 CEBRA-WP 在批量运行中是否真的产生可观察的机制差异。

第 14 页：验证二：84 次运行消融实验

回答的问题：CEBRA-WP 的机制是否可观测，动态机制是否影响高代价调用。

屏幕重点：上方四组策略卡片；左下高代价调用柱状图；右侧三条机制结论。

建议口播：

这一页单独展开 84 次运行实验。实验是四组策略，每组 21 次运行，总计 84 次。四组从左到右代表算法介入程度递增：静态最高分单候选、固定阈值门控、无显式信念状态的动态恢复，以及完整启用轻量信念状态的 CEBRA-WP。

从完成情况看，静态组 21 次全部完成，另外三组各有 1 次失败，所以本文不声称 CEBRA-WP 显著提升成功率。更关键的是机制差异。第一，只有轻量信念状态组 21/21 产生 RuntimeState，说明多维运行时状态可以被持续记录和观察。第二，从高代价调用看，固定阈值门控组是 28 次，动态恢复组和轻量信念状态组都是 20 次，相比固定阈值门控减少 28.6%。这说明动态机制在减少无效高代价调用方面有证据。

可以补充的难点：

这页要诚实说明：lite 和 dynamic 在成功率、高代价调用均值上没有拉开明显差距。lite 的主要证据是状态可观测、预算压力可记录、动作效用可追踪；它证明机制链路成立，但性能增益还需要更大规模、更强压力条件验证。

转场：

有了系统测试和消融实验，最后把贡献和结论边界收束清楚。

第 15 页：贡献、局限与结论边界

回答的问题：本文最后能稳妥地声称什么。

屏幕重点：四项贡献和底部结论边界。

建议口播：

本文贡献可以概括为四点。第一，实现了一个可恢复、可审计的多智能体蛋白质设计工作流原型，打通任务接入、候选计划、工具执行、人工决策和报告输出。第二，提出并实现 CEBRA-WP，将约束、证据、轻量状态和恢复动作纳入规划闭环。第三，用有限状态机、人在环决策、决策记录、快照和事件日志约束智能体行为。第四，用系统测试和 84 次运行消融实验支撑主要结论。

同时，本文的结论限定在工作流层：验证的是规划、执行、恢复和审计机制，不把计算评分扩大为湿实验结论，也不声称 CEBRA-WP 显著提升成功率。候选蛋白的生物学有效性需要后续实验验证。

结束句：

我的汇报到这里结束，恳请各位老师批评指正。

四、备份页使用方式

第 16 页：备份：公式与字段映射

使用场景：老师追问 S_{static} 、 Δ 、 U_{pi} 、 U_a 等符号具体对应什么。

回答要点：

- F_h 对应 `candidate_feasibility`，表示硬可行性过滤，口播名是“候选可执行性检查”。
- S_{static} 对应 `score_breakdown.overall`，表示静态候选效用，口播名是“静态评分”。
- x_t 对应 `RuntimeState`，表示多维运行时状态，口播名是“轻量信念状态”。
- Δ 对应 `runtime_adjustment`，表示有界运行时修正。
- U_{pi} 对应候选运行时效用，表示重排后的候选分数。
- U_a 对应 `ActionUtility`，表示恢复动作效用。

口头补充：

正式页没有展开复杂公式，是因为本文实现采用可解释、可测试的确定性规则和字段映射；备份页用于说明每个理论对象在代码中有对应位置。

第 17 页：备份：蛋白质工具知识图谱如何约束工具链

使用场景：老师追问工具知识图谱的作用，或者问硬过滤依据从哪里来。

回答要点：

- 蛋白质工具知识图谱记录工具能力、输入输出、成本、风险和可用性。
- 实线表示输入/输出兼容链，虚线表示回退路径或降级就绪状态。
- CEBRA-WP 的硬可行性过滤会使用工具、字段模式、输入/输出、成本和风险等信息过滤不可执行候选。

口头补充：

这张图不需要逐个节点讲，只要说明：工具知识图谱把“能不能接上、能不能执行、代价和风险如何”变成确定性约束，避免大语言模型生成看似合理但不可运行的链路。

第 18 页：备份：失败样本与恢复边界

使用场景：老师追问 3 个失败运行说明什么，或者问系统是不是能恢复所有失败。

回答要点：

- 失败样本没有被隐藏，而是保留事件日志和快照。
- 固定阈值门控组可以触发局部修补，但可能出现循环耗尽。
- 轻量信念状态能让运行时状态可观测，但不保证每次都打破局部修补等待循环。
- 无显式信念状态的动态恢复组可以在执行前阻断不可执行候选。

口头补充：

这页说明机制边界，而不是证明所有失败都能恢复。本文更谨慎的结论是：失败路径可记录、可解释、可追溯，部分恢复动作可受控进入人在环决策。

第 19 页：备份：测试证据索引

使用场景：老师追问测试材料和证据在哪里。

回答要点：

- EVD-API 覆盖健康检查、就绪状态、任务详情、事件、报告和待决策对象。
- EVD-TEST 覆盖接口/前端、有限状态机/人在环决策/快照、安全恢复工具和命令行日志。
- FIG-SV 包含仪表盘、任务构建、任务详情和事件时间线截图。
- EVD-LOG 包含事件日志、快照、报告、终止型停止和等待态样本。
- EVD-EXP 包含冒烟运行、完整运行与 84 次运行实验矩阵聚合产物。

口头补充：

证据链主要回答两个问题：系统测试是否覆盖关键路径，运行时恢复决策是否可追溯。

五、常见问题准备

1. 本文的算法贡献是什么？

建议回答：

CEBRA-WP 的论文译名是“约束与证据感知、信念引导、恢复自适应的工作流规划”。它的贡献不在底层蛋白生成，而在运行时工作流规划：先做硬可行性过滤，再用静态评分建立先验排序，最后用运行时状态对合法候选和恢复动作做有界调整。

2. CEBRA-WP 和普通规划器有什么区别？

建议回答：

普通规划器更偏向一次性生成计划。CEBRA-WP 在计划候选之外加入硬可行性过滤、评分分解、轻量信念状态、动作效用和有限状态机/人在环决策审计入口。它能处理执行中的失败、风险和预算变化，而不是只给出初始流程。

3. 为什么不用完整的部分可观测马尔可夫决策过程、强化学习或学习型策略？

建议回答：

本项目的数据量和实验条件不足以支持稳定学习策略，而且毕业设计重点是可解释、可验证的工程机制。因此本文采用轻量信念状态：低维、确定性、可序列化、可写入快照。它不是完整的部分可观测马尔可夫决策过程，也不声称学习到了最优策略。

4. 硬过滤和评分的边界是什么？

建议回答：

硬过滤处理不可违反的条件，例如工具是否存在、字段模式是否合法、输入/输出是否闭合、安全是否阻断、硬预算是否越界。评分只在候选通过硬过滤之后生效，用于比较目标匹配、成本、风险、恢复复杂度和可靠性。安全和预算不能被高分抵消。

5. 模块之间如何传递信息？

建议回答：

通过结构化对象传递，不依赖自然语言计划。规划侧是任务规格、候选集合、候选可执行性字段和评分分解；执行侧回传步骤执行结果、安全检查结果和失败上下文；决策侧留下待决策对象、决策记录、事件日志和快照。

6. 为什么实验里不强调成功率提升？

建议回答：

84 次运行结果没有证明 CEBRA-WP 显著提升成功率，所以本文不把成功率作为主要结论。本文强调的是机制层面的证据：轻量信念状态组 21/21 产生运行时状态，说明状态可观测；动态恢复组和轻量信念状态组相比固定阈值门控组减少高代价调用，说明成本控制有证据；事件日志和快照说明恢复路径可审计。

7. 系统如何保证恢复动作可审计？

建议回答：

恢复动作不会直接调用工具，而是进入待决策对象和决策记录。状态迁移由有限状态机约束，事件日志记录事件，快照保存上下文。系统中断后可以恢复到等待态或已确认状态，避免智能体私自跳过人工确认。

8. 本人工作量如何体现？

建议回答：

不需要单独强调“独自完成”，可以从四个层面体现：一是系统架构和多智能体职责边界；二是 CEBRA-WP 的候选过滤、运行时状态和动作选择机制；三是接口、工作流、适配器、知识图谱、存储和前端工作台的工程实现；四是系统测试和 84 次运行消融实验的验证证据。

六、答辩时避免的表述

- 不说“本文证明了候选蛋白具有湿实验有效性”。
- 不说“CEBRA-WP 显著提升成功率”。
- 不说“轻量信念状态是完整的部分可观测马尔可夫决策过程”。
- 不说“后验目标匹配等价于真实生物学目标”。
- 不把前端展示说成核心贡献，前端只是状态、决策和证据链的可视化入口。